

Residential Society Management & Maintenance Billing System

DBMS Course Project Report

Student Name	Sri Pranav Ramini
Roll Number	25WU0102273
Course	Database Management Systems (DBMS)
Academic Year	2025–2029
Project Type	DBMS Course Project
University	Woxsen University
Branch	CSE - AIML

GitHub Repository: _____

2. Abstract

The Residential Society Management & Maintenance Billing System is a database-driven web application for managing societies, flats, residents, complaints, maintenance bills, payments, maintenance types, and bill details using MySQL.

3. Introduction and Problem Statement

The system organizes society information, flat occupancy, complaints, maintenance billing, and payments in a relational database. Primary keys and foreign keys maintain data relationships and integrity.

4. Objectives and Scope

- Maintain all major society-management records.
- Provide CRUD operations.
- Track bills and payments.
- Manage complaints.
- Generate SQL reports using JOIN, GROUP BY, COUNT and SUM.

5. Software and Hardware Requirements

Category	Requirement
Operating System	Windows / macOS / Linux
Backend	Node.js + Express.js
Database	MySQL
Database Driver	mysql2
Frontend	React + Vite
Browser	Modern web browser
Hardware	8 GB RAM recommended

6. ER Diagram

The ER design contains eight entities: SOCIETY, FLAT, RESIDENT, COMPLAINT, MAINTENANCE_TYPE, MAINTENANCE_BILL, PAYMENT and BILL_DETAIL.

7. Relational Schema — Separate Table for Each Entity

Each entity is shown below in its own completely separate table.

7.1 SOCIETY

Stores residential society information.

Attribute	Data Type	Key / Constraint
Society_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Society_Name	VARCHAR(100)	NOT NULL
Address	VARCHAR(255)	NOT NULL
Registration_No	VARCHAR(50)	NOT NULL, UNIQUE

7.2 FLAT

Stores apartment information and its society relationship.

Attribute	Data Type	Key / Constraint
Flat_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Flat_Number	VARCHAR(20)	NOT NULL
Floor	INT	NOT NULL
Block	VARCHAR(10)	NOT NULL
Type	VARCHAR(20)	NOT NULL
Area	DECIMAL(10,2)	NOT NULL
Society_ID	INT	FOREIGN KEY → SOCIETY(Society_ID)

7.3 RESIDENT

Stores resident information and the flat occupied.

Attribute	Data Type	Key / Constraint
Resident_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Name	VARCHAR(100)	NOT NULL
Phone	VARCHAR(15)	NOT NULL
Email	VARCHAR(100)	NULL
Occupation	VARCHAR(100)	NULL
Flat_ID	INT	FOREIGN KEY → FLAT(Flat_ID)

7.4 COMPLAINT

Stores complaints and their current status.

Attribute	Data Type	Key / Constraint
Complaint_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Complaint_Date	DATE	NOT NULL
Description	VARCHAR(255)	NOT NULL
Status	ENUM('Pending','In Progress','Resolved')	NOT NULL
Flat_ID	INT	FOREIGN KEY → FLAT(Flat_ID)

7.5 MAINTENANCE_TYPE

Stores maintenance categories and rates.

Attribute	Data Type	Key / Constraint
Maintenance_Type_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Type_Name	VARCHAR(100)	NOT NULL
Description	VARCHAR(255)	NULL
Rate	DECIMAL(10,2)	NOT NULL

7.6 MAINTENANCE_BILL

Stores maintenance bills generated for flats.

Attribute	Data Type	Key / Constraint
Bill_ID	INT	PRIMARY KEY, AUTO_INCREMENT

Attribute	Data Type	Key / Constraint
Bill_Date	DATE	NOT NULL
Due_Date	DATE	NOT NULL
Amount	DECIMAL(10,2)	NOT NULL
Status	ENUM('Paid','Pending')	NOT NULL
Flat_ID	INT	FOREIGN KEY → FLAT(Flat_ID)

7.7 PAYMENT

Stores payments made against maintenance bills.

Attribute	Data Type	Key / Constraint
Payment_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Payment_Date	DATE	NOT NULL
Amount_Paid	DECIMAL(10,2)	NOT NULL
Payment_Mode	ENUM('UPI','NEFT','Card','Cash')	NOT NULL
Transaction_ID	VARCHAR(50)	NOT NULL, UNIQUE
Bill_ID	INT	FOREIGN KEY → MAINTENANCE_BILL(Bill_ID)

7.8 BILL_DETAIL

Stores individual maintenance components included in a bill.

Attribute	Data Type	Key / Constraint
Bill_Detail_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Bill_ID	INT	FOREIGN KEY → MAINTENANCE_BILL(Bill_ID)
Maintenance_Type_ID	INT	FOREIGN KEY → MAINTENANCE_TYPE(Maintenance_Type)
Quantity	INT	NOT NULL
Rate	DECIMAL(10,2)	NOT NULL
Amount	DECIMAL(10,2)	NOT NULL

7.9 Entity Relationships

SOCIETY → FLAT → RESIDENT; FLAT → COMPLAINT; FLAT → MAINTENANCE_BILL → PAYMENT;
MAINTENANCE_BILL → BILL_DETAIL ← MAINTENANCE_TYPE.

8. Database and Table Creation

The MySQL database is named Residential_Society_DB and contains the eight relations shown above.

9. Sample SQL Queries and Outputs

9.1 Complaint Count by Status

```
SELECT Status, COUNT(*) AS Number_of_Complaints
FROM COMPLAINT
GROUP BY Status;
```

Output:

```
mysql>
mysql> SELECT
    ->     Status,
    ->     COUNT(*) AS Number_of_Complaints
    -> FROM COMPLAINT
    -> GROUP BY Status;

+-----+-----+
| Status          | Number_of_Complaints |
+-----+-----+
| Resolved        | 12                   |
| Pending         | 7                    |
| In Progress     | 6                    |
+-----+-----+
3 rows in set (0.006 sec)

mysql> |
```

Figure 1. MySQL complaint count output.

9.2 Total Maintenance Amount

```
SELECT SUM(Amount) AS Total_Maintenance_Amount
FROM MAINTENANCE_BILL;
```

Output:

```
mysql>
mysql>
mysql> -- Query 7: Total maintenance amount
Query OK, 0 rows affected (0.002 sec)

mysql>
mysql> SELECT
  ->     SUM(Amount) AS Total_Maintenance_Amount
  -> FROM MAINTENANCE_BILL;
+-----+
| Total_Maintenance_Amount |
+-----+
|                95200.00 |
+-----+
1 row in set (0.009 sec)
```

Figure 2. MySQL total maintenance amount output.

9.3 Number of Residents in Each Flat

```
SELECT F.Flat_Number, COUNT(R.Resident_ID) AS Number_of_Residents
FROM FLAT F
LEFT JOIN RESIDENT R ON F.Flat_ID = R.Flat_ID
GROUP BY F.Flat_ID, F.Flat_Number;
```

Output:

```
mysql>
mysql> SELECT
  ->     F.Flat_Number,
  ->     COUNT(R.Resident_ID) AS Number_o
  -> FROM FLAT F
  -> LEFT JOIN RESIDENT R
  -> ON F.Flat_ID = R.Flat_ID
  -> GROUP BY F.Flat_ID, F.Flat_Number;
+-----+-----+
| Flat_Number | Number_of_Residents |
+-----+-----+
| A101        | 1                   |
| A102        | 1                   |
| A201        | 1                   |
| A202        | 1                   |
| B101        | 1                   |
| B102        | 1                   |
| B201        | 1                   |
| B202        | 1                   |
| C101        | 1                   |
| C102        | 1                   |
| C201        | 1                   |
| C202        | 1                   |
| D101        | 1                   |
| D102        | 1                   |
| D201        | 1                   |
| D202        | 1                   |
| E101        | 1                   |
| E102        | 1                   |
| E201        | 1                   |
| E202        | 1                   |
| F101        | 1                   |
| F102        | 1                   |
| F201        | 1                   |
| F202        | 1                   |
| G101        | 1                   |
+-----+-----+
25 rows in set (0.400 sec)
```

Figure 3. MySQL residents-per-flat output.

10. UI Design and Screenshots

The ResidentCare interface is connected to MySQL and provides dashboard and management screens.

10.1 Dashboard

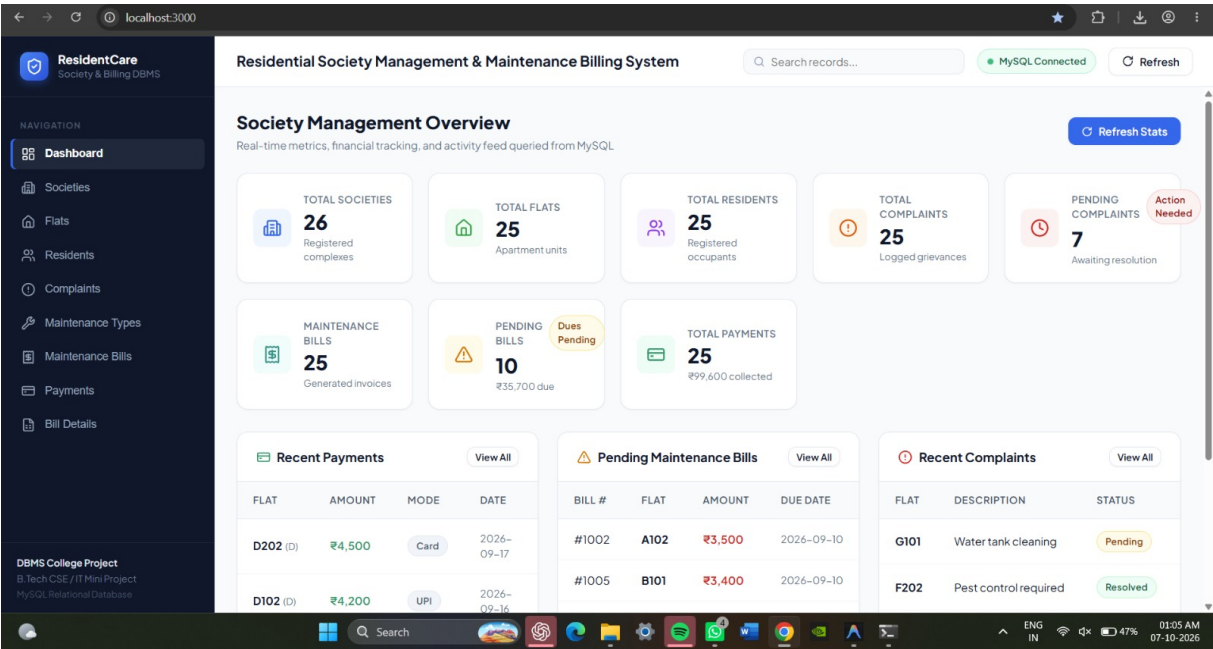


Figure 4. ResidentCare dashboard with MySQL Connected status.

10.2 Payments Module

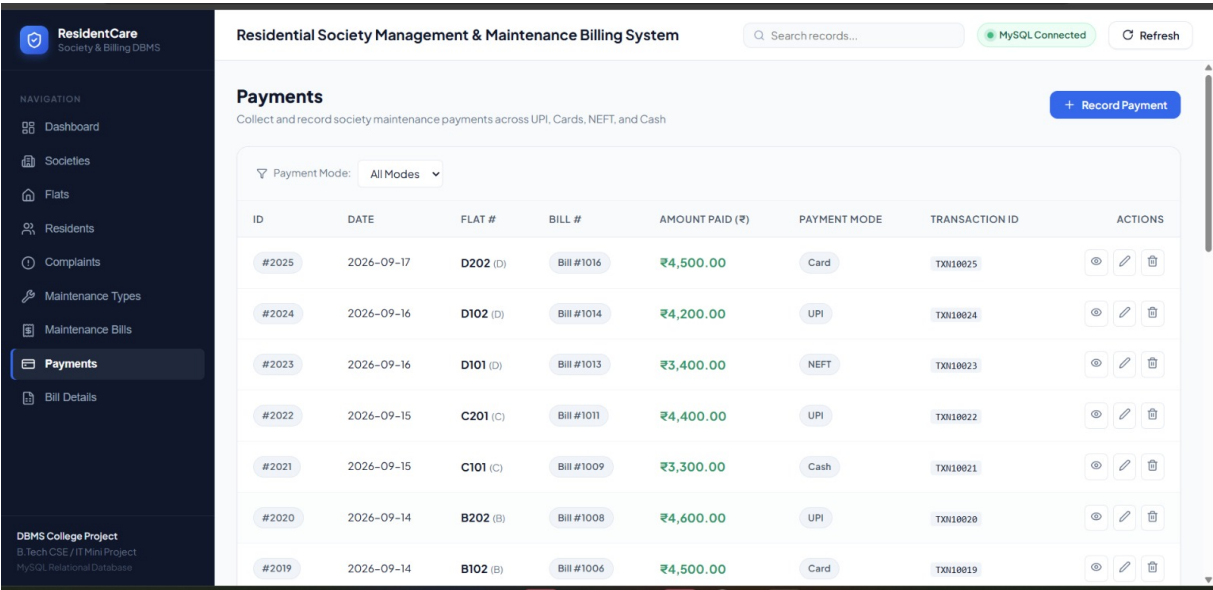


Figure 5. Payments module with records and CRUD actions.

10.3 Flats Module

ResidentCare

Society & Billing DBMS

NAVIGATION

Dashboard

Societies

Flats

Residents

Complaints

Maintenance Types

Maintenance Bills

Payments

Bill Details

DBMS College Project

B.Tech CSE / IT Mini Project

MySQL Relational Database

Residential Society Management & Maintenance Billing System

Search records...

MySQL Connected

Refresh

Flats

Apartment inventory, block assignments, and society associations

+ Add Flat

Filter by Block:

All Blocks

Filter by Type:

All Types

FLAT #	BLOCK	FLOOR	TYPE	AREA (SQ. FT)	SOCIETY NAME (JOIN)	ACTIONS
A101	Block A	Floor 1	2BHK	1,050 sq.ft	Green Valley Apartments	
A102	Block A	Floor 1	2BHK	1,100 sq.ft	Green Valley Apartments	
A201	Block A	Floor 2	3BHK	1,350 sq.ft	Green Valley Apartments	
A202	Block A	Floor 2	2BHK	1,080 sq.ft	Green Valley Apartments	
B101	Block B	Floor 1	2BHK	1,000 sq.ft	Sunrise Residency	
B102	Block B	Floor 1	3BHK	1,400 sq.ft	Sunrise Residency	
B201	Block B	Floor 2	2BHK	1,050 sq.ft	Lake View Society	

Figure 6. Flats module showing society JOIN results.

11. Implementation Details

The application provides navigation for Dashboard, Societies, Flats, Residents, Complaints, Maintenance Types, Maintenance Bills, Payments and Bill Details. Records are retrieved from MySQL and displayed with search, filtering and CRUD-oriented actions.

12. Testing and Results

Database connection: MySQL Connected. Complaint report: grouped status output displayed. Maintenance total: █95,200.00 in the supplied screenshot. Residents-per-flat query: 25 rows in the supplied screenshot. Payments and Flats modules: displayed successfully.

13. Conclusion and Future Enhancements

The system provides a structured relational solution for society management, resident records, complaints, maintenance billing and payments.

14. References

MySQL Documentation; DBMS course materials; project ER diagram and implemented MySQL database; ResidentCare screenshots.

15. Appendix: GitHub Repository Link

GitHub Repository: _____